

Algorithms Analysis and Design

Algorithm:- No singular answer, operational definitions exist, like "Set of steps to reach a goal"

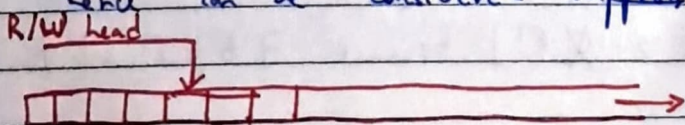
10th Hilbertian Question:- Give an algo to solve a random Diophantine Eqn. [1900]

The definition of an algorithm has answered multiple questions like "What is a machine?", "What is a computer?", etc.

Axioms

1) Information travels at finite speeds. [Sp. Theory of Relativity]

→ Random Access Memory is a myth. All memory is sequential, hence can be considered ~~tappers~~ as tapes.



Speed of the R/W head is 1 cell/per unit time.

2) Finite volume in space can be used to store or retrieve only finite amounts of information reliably [Quantum Mechanics]

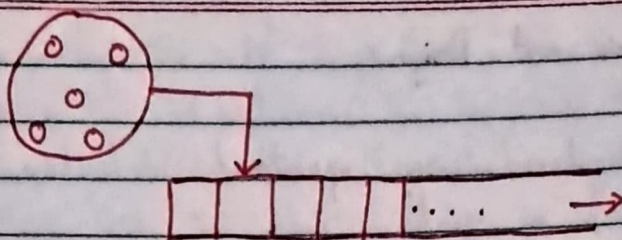
→ This implies that we can not store or precision of irrational numbers. Thus using (perfectly precise) π or the like in an algorithm is not allowed.

Thus each cell can only store a finite amount of data [Based on A1 & A2]

Data in any cell $\in \Gamma$: Finite set of tape alphabet

3) Only finitely many instructions can be packed a priori. [ISA is finite]

→ Computers are finite, hence.



Church Turing Thesis

(set of steps/processes etc)

Algorithm:- Any algorithm that can be simulated using a Turing Machine

Turing Machine:- A Turing Machine is a 7-tuple

$$\langle Q, \Gamma, \delta, q_{\text{start}}, q_{\text{accept}}, q_{\text{reject}}, \epsilon \rangle$$

(q, ϵ) $Q \rightarrow$ Finite state set of states

$\Gamma \rightarrow$ Finite set of state alphabets (that can fill cells)

$$\delta: \Gamma \times Q \rightarrow \Gamma \times Q \times \{L, R\}$$

$$\epsilon \subset \Gamma \quad [\epsilon \notin \Gamma] \text{ because } \exists b \in \Gamma \text{ s.t. } b \notin \epsilon$$

Test

A Turing Machine can have another Turing Machine as its input

Quick

$$U_{TM}(\langle M_{TM} \rangle, x) = M_{TM}(x)$$

brown

For

Where U_{TM} & M_{TM} are Turing Machines x is the input to M_{TM} and $\langle \rangle$ is the string typecast.

Jump

over

ATL

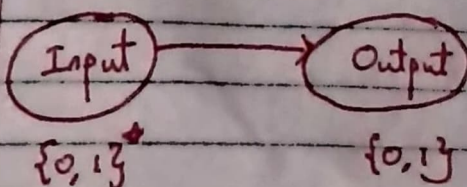
long

Dog

U_{TM} is the Universal Turing Machine

What is a computational problem?

Decision Problems



Problems having multiple bits can be considered as multiple problems having single bit outputs

All problem computational problems are thus considered defined as problems that have a set of inputs which can be bi-partitioned into a T/F inputs. Since the two partitions are ME & E, just being given the input set and one of its subsets is sufficient. All computational problems are hence decision problems.

Decision Problems = (Membership Queries in) Languages.

A language $L \subseteq \{0,1\}^*$

Given a graph G , is it connected or not? Decision problem
(or)

Does G belong to the set of all connected graphs
 $G \in \{G' \mid G' \text{ is a connected graph}\}$ Membership Query

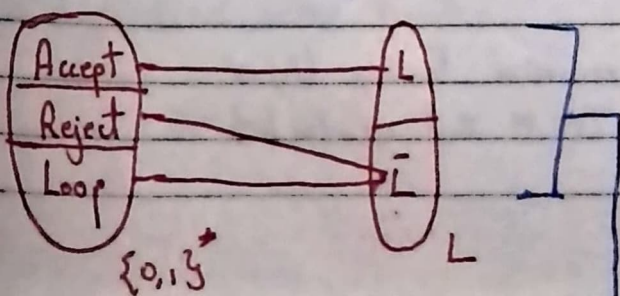
Is a given number n prime?



$n \in \{i \mid i \text{ is prime}\}$

Problems are languages, solutions are Turing Machines

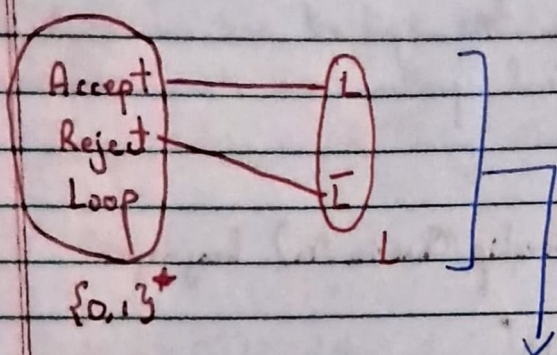
Turing Machines have three states, Accept, Reject and Loop
Languages have only two states, L & $\bar{L}$



Definition:- A language L is said to be recognized by Turing Machine M if:-

$\forall x \in L$; $M(x)$ accepts.
else ; $M(x)$ rejects.

Hence the language L of M $[L(M)] = \{x \mid M(x) \text{ accepts}\}$



Definition:- A language L is said to be decided by TM M is

$\forall x \in L$; $M(x)$ accepts
else ; $M(x)$ rejects

A TM that decides L also recognizes L , but vice versa does not hold.

Theorem:- There are languages that are unrecognizable

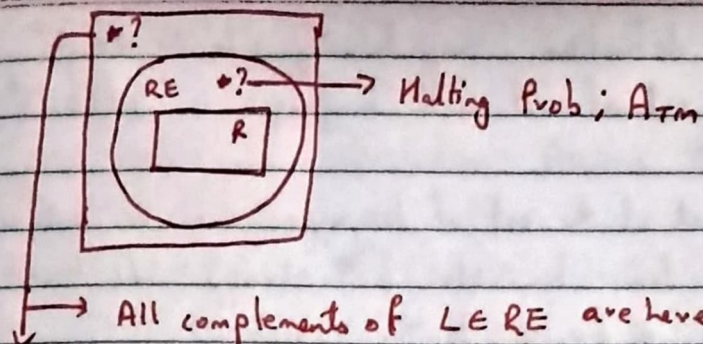
Theorem:- There are languages that are recognizable but undecidable

Defn:- The class of recursively enumerable (RE) languages is

$$RE = \{L \mid \exists \text{ a TM } M \text{ that recognizes } L\}$$

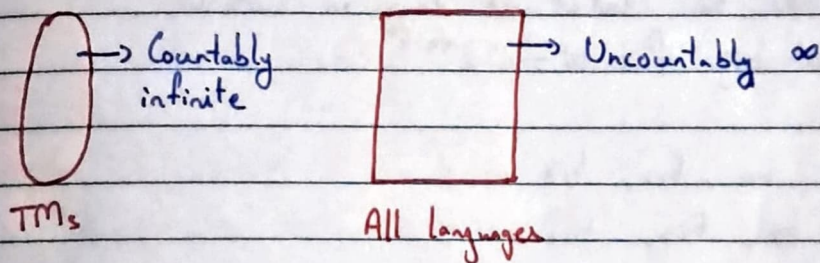
Defn:- The class of recursive languages (R) is

$$R = \{L \mid \exists \text{ TM } M \text{ that decides } L\}$$



How do you show that $\exists$ languages here?

Take the set of all TMs and the set of all problems



Any TM can be stored in a machine with finite bit strings, hence the no. of TMs $\leq$ no. of finite len bin strings
Thus $TMs \subseteq \{0,1\}^*$ $\rightarrow$ They can be enumerated
How?

Theorem:- $\{0,1\}^*$ is countable

Proof:- $f: \mathbb{N} \rightarrow \{0,1\}^*$ must be a bijection

Consider the following enumeration of $\{0,1\}^*$

Shorter first, lexicographic ordering for eq lens.

$\epsilon, 0, 1, 00, 01, 10, 11, \underbrace{000, \dots}_8, \underbrace{0000, \dots}_{16}, \dots$

This is a bijective map cause every string has a fixed index.
[Find a closed order formula]

Theorem:- The set of languages is uncountable.

Proof:- By definition, any language $L \subseteq \{0,1\}^*$.
So the set of all such languages is $\mathcal{P}(\{0,1\}^*)$

Each subset of the set of languages can be uniquely identified by an ∞ len bin str $[str[i] = 0/1]$ based on whether the i^{th} element $\in$ or $\notin$ to the subset

Any $L \equiv$ ∞ len bin str.

Assume that the set of all langs is countable.
Then $\exists$ a bij $f: \mathbb{N} \rightarrow 2^{\{0,1\}^*}$

$$f(1) = b_{11}, b_{12}, b_{13}, \dots$$

$$f(2) = b_{21}, b_{22}, b_{23}, \dots$$

$$f(3) = b_{31}, b_{32}, b_{33}, \dots$$

Since it's countable, all $f(i)$ have been mapped to these $f(i)$

Now we diagonalization to create a new language L

$$L = \overline{b_{11}}, \overline{b_{22}}, \overline{b_{33}}, \overline{b_{44}}, \dots$$

This L has no mapping and is distinct from all $f(i)$ in at least 1 pos. Hence it's uncountably ∞ .

The class RE are those programs which give yes, no or loop, but never returns a wrong answer.

Given a TM M and input x , will M accept x ?

$$A_{TM} = \{ \langle M, x \rangle \mid M \text{ accepts } x \}$$

This problem is recognizable, not decidable $\rightarrow \in RE$

$\langle M \rangle \rightarrow$ Program Typecast to a string to be used as input!

Date: _____

Theorem:- A_{TM} is undecidable.

Proof:- We prove by contradiction. Assume that A_{TM} was decidable.

$\rightarrow \exists$ a TM H that decides A_{TM} .

$\rightarrow \forall \langle M, x \rangle \in L, H(\langle M, x \rangle)$ accepts;

$\forall \langle M, x \rangle \notin L, H(\langle M, x \rangle)$ rejects; [Not loop]

Consider a TM D which does the following

i) On input M

① Run H on $(\langle M, M \rangle)$ $H(M, \langle M \rangle)$

② Return $\neg H$'s result (accept on reject and vice versa)

Execute $D(\langle D \rangle)$

On input $\langle D \rangle$, Run $H(D, \langle D \rangle)$

If H accepts, then reject $\rightarrow (D, \langle D \rangle) \notin A_{TM}$

This implies D accepts $\langle D \rangle$ then D rejects.

Contradiction

If H rejects, D accepts $\rightarrow (D, \langle D \rangle) \in A_{TM}$

This implies D rejects $\langle D \rangle$ then D accepts.

Contradiction

(or)

	$\langle M_1, M_1 \rangle \dots \langle M_i \rangle$
M_1	$H(M_1, \langle M_2 \rangle)$
M_2	
$\vdots$	
M_i	

Use diagonalization create a new D , but $H(D, \langle D \rangle)$ is a contradiction.

- That's how diagonalization works! $\Leftarrow$

Theorem:- If $L \in RE$, and $\bar{L} \in RE$, then $L \in R$

Proof:- $L \in RE \Rightarrow \forall x \in L, \exists \text{ TM } M \text{ s.t. } M \text{ accepts } x$
 $\neg \forall$

i) $L \in RE \Rightarrow \exists \text{ a TM } M \text{ s.t.}$

$\forall x \in L; M \text{ accepts } x$

$\forall x \in \bar{L}; M \text{ rejects / loops on } x$

ii) $\bar{L} \in RE \Rightarrow \exists \text{ a TM } M' \text{ s.t.}$

$\forall x \in \bar{L}; M' \text{ accepts } x$

$\forall x \in L; M' \text{ rejects / loops on } x$

Create a decider ~~that~~ by running M and M' in parallel.

$\bullet \bullet M \text{ accepts } x \in L \text{ and } M' \text{ accepts } x \in \bar{L}.$

Theorem:- ~~$A_{TM} \in RE$~~ $\bar{A}_{TM} \notin RE$

Proof:- $A_{TM} \in RE$; $A_{TM} \notin R$

By contradiction and prev theorem, $\bar{A}_{TM} \notin RE$.

Time (n^k) $\rightarrow$ Problems solved in $O(n^k)$, where $k \in \text{Rational}$

Time ($n^{k \in \mathbb{Q}}$) $\rightarrow$ Superset of Time (n^k)

Reduction:- A language L_1 is mapping reduced to L_2 if
 $\exists$ a computable fn $f: \{0,1\}^* \rightarrow \{0,1\}^*$ s.t. $\downarrow x \in L_1 \Rightarrow f(x) \in L_2$
 $\forall x \in \{0,1\}^*$

$L_1 \leq_m L_2$

Defn:- Language $A \leq_m B$

$$f(\langle M, x \rangle) = \{ H(M, x) \wedge [M(x)] \}$$

Examples:-

Let $HALT_{TM} = \{ \langle M, x \rangle \mid M \text{ halts on input } x \}$

Lemma:- $A_{TM} \leq_m HALT_{TM}$

Proof:- Decider for A_{TM} (using the one for $HALT_{TM}$)
Let H decide $HALT_{TM}$

On input $\langle M, x \rangle$:-
 → Run $H(M, x)$
 → If H rejects, REJECT
 → If H accepts, ACCEPT or REJECT based off $M(x)$

Theorem:- If $A \leq_m B$ and $B \in R \rightarrow A \in R$

Theorem:- If $A \leq_m B$ and $A \notin R \rightarrow B \notin R$

Completeness Theory:-

Suppose $\forall A \in RE, A \leq_m HALT_{TM}$.
Then $HALT_{TM}$ is RE-complete

Theorem:- $HALT_{TM}$ is RE-complete

→ Anything in RE

Proof:- Let TM M recognize A & H decides $HALT_{TM}$

Construct a decider for A .

On input x :

Run $H(M, x)$. If



$$EQUAL_{TM} = \{ \langle m_1, m_2 \rangle \mid L(m_1) = L(m_2) \}$$

Theorem:- $EQUAL_{TM}$ is unrecognizable

Proof:- $\overline{A_{TM}}$ is unrecognizable ($\overline{A_{TM}} \notin RE$)

Let E recognize $EQUAL_{TM}$

w

M_1 : On input ~~x~~ , Reject

M_2 : On input w , if $x \neq w$ Reject

If M accepts x , ACCEPT.

Else Reject

If $E(m_1, m_2)$ accepts $\Rightarrow M$ does not accept x .

$(M, x) \in A_{TM}$

Divide and Conquer.

Mergesort, Binary Search, Median of Medians.

Fourier Transforms:-

Input:- Two polynomials $A(x)$ & $B(x)$ of degree $n-1$

Output:- Their product $C(x) = A(x) \cdot B(x)$

$A[n]$ $\rightarrow$ coefficients of A

$B[n]$ $\rightarrow$ coefficients of B

$C[2n-1]$ $\rightarrow$ coefficients of C

$$A(x) = \sum_{i=0}^{n-1} a_i x^i$$

$$B(x) = \sum_{i=0}^{n-1} b_i x^i$$

$$C(x) = \sum_{i=0}^{2n-1} c_i x^i$$

$$c_i = \sum_{j=0}^i a_j b_{i-j} \mapsto \text{But this is an } O(n^2) \text{ algorithm}$$

But Fast Fourier Transforms can do it in $O(n \log n)$

Algorithm :-

- $O(n)$ $\leftarrow$ i) Evaluate Select $2n-1$ or more points $x_1, \dots, x_m$ [$m \geq 2n-1$]
 ii) Evaluate $A(x_i)$ and $B(x_i)$ $\mapsto O(n^2)$ [n pts with $n-1$ substitutions per pt]
 iii) Multiply $A(x_i) \times B(x_i) \Rightarrow C(x_i) \mapsto O(n)$
 iv) Interpolate $C(x_i)$ to obtain $C(x) \mapsto O(n^2)$

Let $A(x)$ be the polynomial

$$A(x) = A_e(x^2) + x A_o(x^2)$$

$$A(x) = 5x^4 - 3x^3 + 7x^2 + x + 1$$

$$A_e(x) = 5x^2 + 7x + 1$$

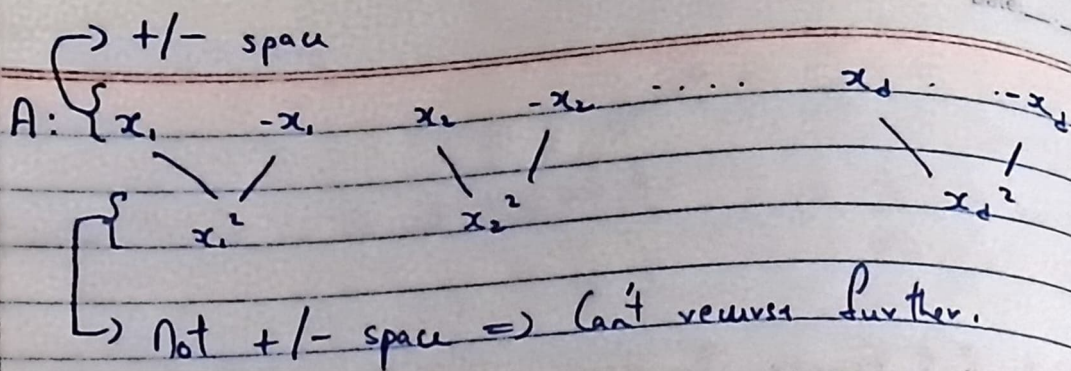
$$A_o(x) = -3x + 2$$

$$\text{For any } A(x_i) = A_e(x_i^2) + x_i A_o(x_i^2)$$

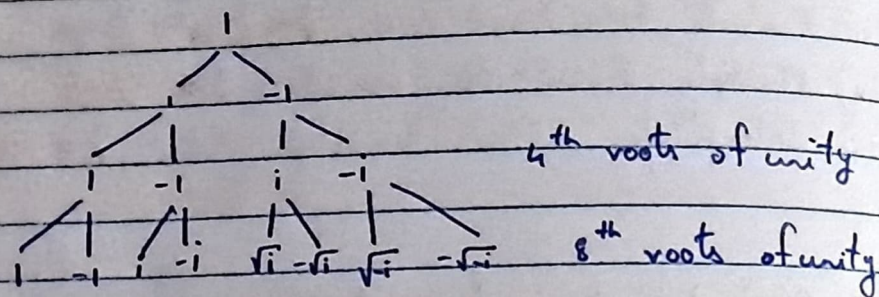
$$A(-x_i) = A_e(x_i^2) - x_i A_o(x_i^2)$$

$$T(n) = 2T(n/2) + O(n)$$

~~$$\text{But } A_e(x_i^2) = A_e(x_i^4) + x_i^2 A_o(x_i^4)$$~~
~~$$A_e(-x_i^2)$$~~



Hence, we pick complex no.s to evaluate.
Let begin from the bottom.



m^{th} roots of unity

$$e^{i \frac{2\pi}{m} j}$$

$$j \rightarrow [1, m]$$

m is an exact power of 2 $\geq 2n-1$

Goal: To evaluate a polynomial A on the m^{th} roots of unity
 $1, \omega, \omega^2, \dots, \omega^{m-1}$

Input: $A[]$

Output: $[A(1), A(\omega), \dots, A(\omega^{m-1})]$

$$A = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{bmatrix} \quad A(x) = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^d \\ 1 & x_2 & x_2^2 & \dots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{d+1} & x_{d+1}^2 & \dots & x_{d+1}^d \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{bmatrix}$$

To evaluate our output, we multiply A with

$$\begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^d \\ \vdots & & & & \\ 1 & \omega^{m-1} & \omega^{2(m-1)} & \dots & \omega^{(m-1)^2} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{m-1} \end{bmatrix} = \begin{bmatrix} A(1) \\ A(\omega) \\ \vdots \\ A(\omega^{m-1}) \end{bmatrix}$$

$\hookrightarrow i, j^{\text{th}}$ entry $\rightarrow \omega^{ij} \quad \forall i, j \in [0, m-1]$

$\hookrightarrow$ Discrete Fourier Transform.

~~FFT~~ $\text{FFT}(A, \omega)$:

if $\omega == 1$:

return $A(1)$

else:

$A_e = [A[0], A[2], \dots]$

$A_o = [A[1], A[3], \dots]$

~~$\text{FFT}(A_e)$~~

$E = \text{FFT}(A_e, \omega^2)$

$O = \text{FFT}(A_o, \omega^2)$

~~for i in range $(m-1)$:~~

~~$A[i] = E[i/2] + \omega^2 O[i/2]$~~

~~# E and O are half the size of A , hence we~~

~~# must read at half the index~~

for i in range $(m/2)$:

$A[i] = E[i] + \omega^0 O[i]$

$A[m/2+i] = E[i] - \omega^0 O[i]$

$\blacksquare$ Check penultimate page for the formula

Now, the second step in our algorithm (check ^{antigenultindr} page) runs in $O(n \log n)$

In step 1, we pick $m \geq 2n-1$ where m is a power of 2, which are just the m^{th} roots of unity.

I'm lost ~~uuu~~ [We've Fucked]

Interpolation

$$\begin{bmatrix} M \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{bmatrix} = \begin{bmatrix} c(1) \\ c(\omega) \\ \vdots \\ c(\omega^{m-1}) \end{bmatrix}$$



$$M_{ij} = \omega^{ij}$$

$$\begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{bmatrix} = M^{-1} \begin{bmatrix} c(1) \\ c(\omega) \\ \vdots \\ c(\omega^{m-1}) \end{bmatrix}$$

$$\cancel{M}^{-1} = \frac{1}{m} M(\omega^{-1}) \quad \begin{bmatrix} \text{Yum} & \text{Yum} \end{bmatrix}$$

or
[Yum]*

$$\begin{bmatrix} \omega^{ij} \end{bmatrix}_{m \times m} \times \begin{bmatrix} \omega^{-ij} \end{bmatrix}_{m \times m}$$

$$= \begin{bmatrix} m & & & \\ & m & & \\ & & m & \\ & & & \ddots \end{bmatrix}$$

This (somehow! some lucky how!) makes the last step $O(\log n)$

Karatsuba's Algorithm:-

Large integer multiplication [Two n -bit +ve integers]

Normal multiplication takes $O(n^2)$

Fastest way to multiply large integers is still FFT by converting the no. into a polynomial $f(x)$ where $f(2) = \text{int.}$

Sorting Detour:-

Assuming all numbers are bounded.

Countsort :- Store ~~them~~ in an array at loc x_i
 $\downarrow$
 $\text{count}(x_i)$

Radix sort:- Apply countsort on every position, units, tens, etc. until the largest place.

End detour:-

Multiplying 2 complex numbers $(a+ib)(x+iy)$

$$\Rightarrow ax - by + i(bx + ay)$$

Looks like it needs 4 real products, but one can just use 3:-

$$(i) \quad p_1 = (x_1 + y_1)(x_2 + y_2)$$

$$(ii) \quad p_2 = x_1 \cdot x_2$$

$$(iii) \quad p_3 = y_1 \cdot y_2$$

Similarly, for n bit numbers a & b

$$a = a_L \cdot 2^{n/2} + a_R$$

$$b = b_L \cdot 2^{n/2} + b_R$$

$$ab = a_L b_L 2^n + 2^{n/2} (a_L b_R + a_R b_L) + a_R b_R$$

We can use this to establish a recurrence, but it would be $O(n^2)$. Instead we apply Gauss' Logic, reducing it to 3 multiplications:-

$$p_1 = (a_1 + a_2)(b_1 + b_2)$$

$$p_2 = a_1 b_1$$

$$p_3 = a_2 b_2$$

$$\text{and } ab = p_2 2^{n/2} + 2^{n/2} [p_1 - p_2 - p_3] + p_3$$

Recurrence on this results in

$$\begin{aligned} T(n) &= 3 T(n/2) + O(n) \\ &= O(\log O(n \log^3)) \end{aligned}$$

$$\begin{aligned} T(n) &= 3 T(n/2) + O(n) \\ &= 3 [3 T(n/4) + O(n/2)] + O(n) \\ &= 3^2 (T(n/2^2)) + 3 \cdot n/2 + n \\ &= \dots \\ &= 3^i T(n/2^i) + 3^2 \cdot n/2^2 + 3 \cdot n/2^1 + n \\ &= 3^i T(n/2^i) + n \sum_{j=0}^{i-1} 3^j / 2^j \end{aligned}$$

$$\text{Let } i = \log_2 n$$

$$T(n) = 3^{\log_2 n} + n \sum_{j=0}^{\log_2 n - 1} 1.5^j$$

$$\approx O(n \log^3)$$

Strassen's Algorithm:-

$$\begin{bmatrix} A \end{bmatrix}_{n \times n} \begin{bmatrix} B \end{bmatrix}_{n \times n} \rightarrow \begin{bmatrix} A & B \\ C & D \end{bmatrix}_{n \times n} \begin{bmatrix} E & F \\ G & H \end{bmatrix}_{n \times n}$$

$$= \begin{bmatrix} AE+BG & AF+BH \\ CE+DG & CF+DH \end{bmatrix}_{n \times n}$$

$$T(n) = 8 T(n/2) + \underline{O(n^2)} \rightarrow \text{Addition}$$

$$= 8 [8 T(n/4) + O((n/2)^2)] + O(n^2)$$

$$= \dots$$

$$\approx O(n^3)$$

BAD! :<

Strassen's ~~technique~~ technique :- He does it in 7 multi

$$T(n) = 7(T(n/2)) + O(n^2)$$

$$= \dots$$

$$= O(n^{\log_2 7})$$

The product would then be

$$\left[\begin{array}{c|c} P_5 + P_6 + P_4 - P_2 & P_1 + P_2 \\ \hline P_3 + P_4 & P_5 - P_7 + P_1 - P_3 \end{array} \right]$$

$$P_1 = A(F-H)$$

$$P_2 = (A+B)H$$

$$P_3 = (C+D)E$$

$$P_4 = D(G-E)$$

$$P_5 = (A+D)(E+H)$$

$$P_6 = (B-D)(G+H)$$

$$P_7 = (A-C)(E+F)$$

Fibonacci?

$$F_n = F_{n-1} + F_{n-2}$$

$$F_0 = 0$$

$$F_1 = 1$$

Recursive $\rightarrow 2^n \rightarrow$ bad :<

we have $O(n)$ additions.

Iterative $\rightarrow n \rightarrow$ okay

$\hookrightarrow$ but for large n , add becomes $O(n)$

$\hookrightarrow \underline{\underline{n^2}}$ for large n

$$\begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

$$\begin{pmatrix} F_2 \\ F_3 \end{pmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^2 \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

$$\vdots$$

$$\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

$\hookrightarrow$ logn multiplications

Greedy Algorithms

Used in optimization problems

Ex :- Shortest path - single source, Huffman encoding.

Optimum Substructure Property:- ~~Sub~~ Subparts of a problem must also be using the most efficient solution.

If $S \rightarrow V$ is the shortest path, then $S \rightarrow T \Rightarrow S \rightarrow V + V \rightarrow T$.

In such cases we usually reverse, given the first step to be taken.

Greedy - Choice Property :- Picking the right first step.

We need both Optimum Substructure and Greedy Choice to have an efficient Greedy Algo

If only the Optimum Substructure holds, we try out all possibilities, but this is slow unless we have :-

Subproblems

Overlapping Substructure Property: If we are able to create a DAG with the subproblems. We topological sort first and solve subproblems in toporder

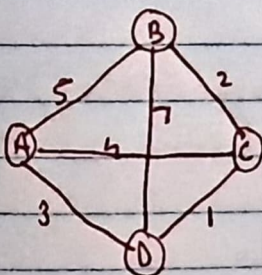
Greedy is a spl case of DP where the DAG is a path.

Minimum Spanning Trees:-

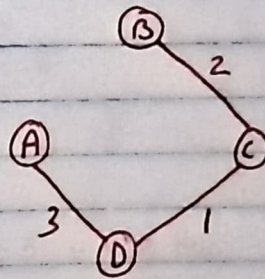
Input:- Edge Weighted Simple Undirected Graph.

Subgraph of a tree on all its vertices

Cost of tree = $\sum$ edge wts.



Cost := 21



Cost = 6 $\rightarrow$ MST

Output:- Least cost Spanning Tree T of G .

The Cut Property:-

Let $G = (V, E)$.

Suppose $X \subseteq E$ belongs to some MST of G .
Then let V be divided into S and V/S such that
no edge spans from $S \rightarrow V/S$ or vice versa.

Then the least cost edge e b/w S & V/S is s.t.
 $X \cup \{e\}$ is part of some MST of G .

Algo:- [Kruskal]

Sort edges by wt.

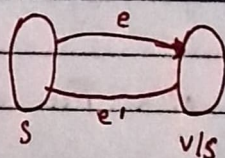
Add edges in that order until unless cycle is formed
the MST

Proof:- Suppose $X \cup \{e\}$ is part of T if X is a part of T .

$$e \notin E(T)$$

$$e' \in T \rightarrow e' \in \text{cut}$$

$$\text{wt}(e) \leq \text{wt}(e')$$



$(T \cup \{e\}) \setminus \{e'\}$ has to be connected cause
 $T \cup \{e\}$ formed a cycle.

$\hookrightarrow$ had n edges, so the graph $(T \cup \{e\}) \setminus \{e'\}$
has $n-1$ edges and is a connected $\rightarrow$ is a tree T'

$$\text{cost}(T') = \text{cost}(T) + \text{wt}(e) - \text{wt}(e').$$

$$\leq \text{cost}(T)$$

Since T is an MST, $\text{cost}(T') = \text{cost}(T)$

Hence T' is an MST

Prim's Algorithm:-

let $x = \emptyset$

Repeat:

- (i) Find a cut that doesn't contain x
- (ii) Update $x = x \cup \{e\}$ where e is the min-cost edge in that cut.

Until $|V|-1$ edges are added.

Kruskal's is preferred because of advanced data structures like union-find :- Find $\rightarrow O(\log n)$, Union $\rightarrow O(1)$ and tot complexity of $O(|E| \log |V|)$ without path compression.

With path compression $\Rightarrow$ Find $\rightarrow O(\log^* n)$

Activity Selection / Interval Scheduling Problems:

Input = $\{ [s_1, f_1), [s_2, f_2), \dots, [s_n, f_n) \}$

Non overlapping interval:

Output:- λ Maximum sized subset of compatible activities

Hint:- Activity A_i is part of some optimum soln.
Remove all activities compatible with A_i .

Greedy Choice:- The activity with the least time finish time is always a part of some optimum soln.

Algo:- Sort acc to finish time. Initialize the first activity.
Pick next activity ~~and~~ that is compatible and continue

Knapsack Problem:-

Input:- Collection of items $\{(c_1, w_1), (c_2, w_2), \dots, (c_n, w_n)\}$

Output:- Subset of items s.t. total wt $\leq W$, maximize cost

Matroid Problem:-

Definition:- A matroid is a pair $\langle S, I \rangle$ where:

(a) S is a finite non-empty set

(b) I is a family of collection of subsets of S s.t. $[\emptyset \in I]$

set of independent sets $\downarrow$

(i) Hereditary Property:- If $A \subseteq S$ belongs to I , then $\forall B \subseteq A, B \in I$

(ii) Exchange Property:- If $A, B \in I$ and $|B| > |A|$, then $\exists x \in B \setminus A$ s.t. $A \cup \{x\} \in I$

Q

Q Question:- Given a Matroid $\langle S, I \rangle$ and given non-ve int wts to elements of S , find max wt element of I (or subset of S)

Q

Q

Q

$$w(A) = \sum_{x \in A} w(x) \quad \cdot \quad x \in S \text{ \& } w(x) = \text{wt of } x.$$

Ans:- Universal Greedy Algo for weighted matroids:

→ Sort the elements of S in non-increasing order of wts

→ $A \leftarrow \emptyset$

→ In that order, let x be the next element if $A \cup \{x\} \in I$ then $A \leftarrow A \cup \{x\}$

Finally output A .

Proof of correctness/optimality

(i) Greedy Choice:- The element $x \in S$ with max wt belongs to some optimum solution.

Proof:- Let $A \in I$ be an optimum soln.

if $x \in A$:

pass

else: $[x \notin A \wedge \{x\} \in I]$

Let $A = \{a_1, a_2, \dots, a_k\}$

From exchange property, we know that $\exists$ a $B = (A \cup \{x\}) \setminus \{a_i\}$
 $\wedge B \in I$

$$w(B) = w(A) + w(x) - w(a_i) \geq w(A)$$

a_i : has lesser lesser wt than x cause x appeared before
 a_i in sorted order

$M = \langle S, I \rangle$ contracts to $M' = \langle S', I' \rangle$, $A = A' \cup \{x\}$
 $S' = S - \{x\}$; I' : sets of I with x erased

$$I' = \{B \mid B \cup \{x\} \in I\}$$

$$w(A) = w(A') + w(x)$$

Given an edge weighted graph $G = (V, E)$

Let $S_G = E$

$$I_G = \{E' \subseteq E \mid E' \text{ does not have a cycle}\}$$

Theorem:- $M_G = \langle S_G, I_G \rangle$ is a matroid.

I_G is hereditary, cause if $A \in I_G$, all $S \subseteq A$ also $\in I_G$.

T_g has the exchange property.

$$|E_1| > |E_2|$$

↓ $\rightarrow$ forest with $|V| - |E_2|$ trees
 forest with
 exactly $|V| - |E_1|$
 trees

E_1 has an edge (u, v) s.t u, v are in diff trees in E_2 .
 Add this edge to E_2 , which will not form a cyclic graph.

$$E_2 \in T \rightarrow E_2 \cup \{(u, v)\} \in T_g$$

Hence M_g is a matroid.

I_q has the exchange property.

$|E_1| > |E_2|$
 $\downarrow$
 forest with
 exactly $|V| - |E_1|$
 trees $\rightarrow$ forest with $|V| - |E_2|$ trees

E_1 has an edge (u, v) s.t. u, v are in diff trees in E_2 .
 Add this edge to E_2 , which will not form a cyclic graph.

$$E_2 \in I \rightarrow E_2 \cup \{(u, v)\} \in I_q$$

Hence M_q is a matroid.

Dynamic Programming :-

A bag of subproblems s.t. it forms a DAG of smallest to largest

Shortest Path in a DAG.

Input: Weighted Directed Acyclic Graph with source s and dest D

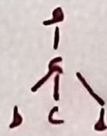
Output: Shortest path from s to D

Algo: Topo-sort the DAG

$$P(s, D) = \min(P(s, x) + dP(x, D)) \quad \forall x \text{ in paths b/w } s \rightarrow D$$

$$P(s, s) = 0$$

Recurse



apsara

Longest Path?

Algorithm :- Toposort the DAG

$$P(s, i) = \max (P(s, u) + d(u, i)) + u$$

$$P(s, s) = 0 \text{ \& return}$$

Both run in $\Theta(E)$ $O(V+E)$

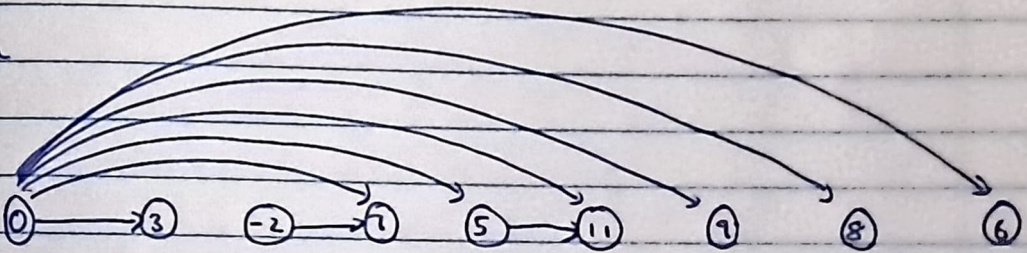
Longest increasing subsequence [LIS]

Input :- Array of nos

Output :- LIS $\rightarrow$ Strictly increasing

Subsequence $\rightarrow$ any no. can be erased, but order is fixed.

Ex



Similarly connect all nodes only to those nodes strictly greater than itself

Now find the longest path in the DAG

$$L(i, j) = \max_{(u, j) \in E} (1 + L(i, u))$$

Find the ~~longest~~ LIS for all nodes, but the nodes ~~are~~ appearing

Complexity = $O(h \cdot |E|)$ $\rightarrow$ h is no. of source nodes
 $\uparrow$ $\hookrightarrow [a, b, \dots, n]$

in any prob LIS cannot be the starting node for another LIS because of greedy choice and optimum substructure.

Edit Distance

min no. of edit operations (insert, delete, overwrite) to convert one str to another.

Ex- EARTH & HEART

- (i) insert H to the first
- (ii) delete H from the last

SUNNY & SNOWY

- (i) Overwrite U $\rightarrow$ N
- (ii) " N $\rightarrow$ O
- (iii) " N $\rightarrow$ U

Q) Gives a currency $D = \{d_1, d_2, \dots, d_n\}$ and a target V
find the minimum set of coins.

Algo:- Init an empty sol set
Sort D by in desc
Find the largest $d_i \in D, d_i \leq V$
sol set = sol set $\cup \{d_i\}$
 $V \leftarrow V - d_i$
repeat.

Optimality:-

Assume our algorithm is not optimal.

Say our algorithm differs with the optimal soln at some position j . But at position j , our $g_j > O_j$ because of greedy choice. In every case, our optimal replacing a coin

in our greedy soln results in an optimal soln O where $19/5/01$.
Hence, the greedy soln is not optimal.

Q) Interval Scheduling:

Init empty array.

Sort input array by finish time ~~asc~~ & in case of ties by start time.

Pick first element and append to array.

Iterate and pick next interval with ~~start~~ $s_i \geq f_{i-1}$

Repeat till done.

Proof:-

Base Case:- When $A = \emptyset$, all jobs in A are compatible.

Induction Hypothesis:- All jobs in $i < j$ in A are compatible.

Induction Step:- ~~We add $i=j$ if j in A , & all jobs~~
 $i < j+1$ in A are compatible. else all jobs
in $i < j+1$ in A are compatible.

Edit Distance:-

$$X = x_1 x_2 \dots x_n$$

$$Y = y_1 y_2 \dots y_m$$

Add as many blanks as necessary to make both of same length and such that the strings are aligned.
Cost of alignment = no. of mismatches

Subproblems:-

(prefix) 1st i chars of X

(prefix) 1st j chars of Y

We call $Edit(i, j)$ To try and reach the goal of $Edit(r, m)$

(a) $x_i \rightarrow Edit(i, j) = Edit(i-1, j) + 1$

(b) $y_j \rightarrow Edit(i, j) = Edit(i, j-1) + 1$

(c) $x_i, y_j \rightarrow Edit(i, j) = \begin{cases} Edit(i-1, j-1) & ; x_i = y_j \\ Edit(i-1, j-1) + 1 & ; \text{else} \end{cases}$

$$Edit(i, j) = \min \begin{pmatrix} Edit(i-1, j) + 1 \\ Edit(i, j-1) + 1 \\ Edit(i-1, j-1) + \text{diff}(x_i, y_j) \end{pmatrix}$$

$Edit(0, j) = j$
 $Edit(i, 0) = i$ } Base Case

	x_1	x_2	x_3	...	x_n
y_1					
y_2					
y_3					
...					
y_m					
					$Edit(m, n)$

Fill row or column wise (because for any cell, the cells to the left, above and diagonally up and left must be filled)

	H	E	A	R	T	
	0	1	2	3	4	5
E	1	1	1	2	3	4
A	2	2	2	1	2	3
R	3	3	3	2	1	2
T	4	4	4	3	2	1
H	5	4	5	4	3	2

Blue Arrows show one possible way of deriving cell values, row wise

H 5 → 4 → 5 → 4 → 3 → 2 → E(n,m)

Q) Edit Distance b/w POTYNO POLYNOMIAL & EXPONENTIAL

	P	O	L	Y	N	O	M	
	0	1	2	3	4	5	6	7
E	1	1	2	3	4	5	6	7
X	2	2	2	3	4	5	6	7
P	3	2	3	3	4	5	6	7
O	4	3	2	3	4	5	5	6
N	5	4	3	3	4	4	5	6
E	6	5	4	4	4	5	5	6
N	7	6	5	5	5	4	5	6
T	8	7	6	6	6	5	5	6

Chain Matrix Multiplication:-

Input:- Chain of matrices

$$M_{p_0 \times p_1} \times M_{p_1 \times p_2} \times M_{p_2 \times p_3} \times \dots \times M_{p_{n-1} \times p_n}$$

$$\text{Output:- } M_{p_0 \times p_n} = \prod_{i=1}^n M_{p_{i-1} \times p_i}$$

$M_{pq} \times M_{qr} \Rightarrow pqr$ multiplications

Example:- $M_{100 \times 10}^{(1)} \times M_{10 \times 1000}^{(2)} \times M_{1000 \times 1}^{(3)} \times M_{1 \times 100}^{(4)}$

no. of mult $\Rightarrow 100 \times 10 \times 1000 + 1000 \times 1 \times 100 + 100 \times 1000 \times 100$
 $\Rightarrow 11100000$

(v) $(1(2 \times 3))4 \Rightarrow 10^3 + 10^4 + 10^4 = 21000$

Q) How many different ways are there to parenthesise a chain of n matrices?

Catalan Numbers $= \frac{1}{n+1} {}^{2n}C_n$

$C(i, j) \Rightarrow M_i \times M_j \rightarrow M_{p_{i-1} \times p_i} \times M_{p_i \times p_{j+1}}$

Goal:- To find $C(i, n)$

Easy step:- $C(i, i+1) = p_{i-1} \times p_i \times p_{i+1} \parallel C(i, i) = 0$

Suppose $M_1 \times M_2 \times \dots \times M_k$

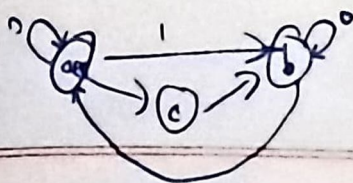
$C(i, j) = \min_{i < k < j} \{ C(i, k) + C(k+1, j) \}$
 p_i, p_k, p_j

	1	2	3	4
1	0	10^4	10^5	10^6
2		0	10^1	10^2
3			0	10^5
4				0

Goal

$C(3) = \min \left\{ \begin{aligned} &C(1, 2) + C(3, 3) + p_0 p_2 p_3 \\ &C(1, 1) + C(2, 3) + p_0 p_1 p_3 \end{aligned} \right.$

Similarly



Knapsack :-

$$f(w, i) = \begin{cases} f(w - a_i, i+1) & a_i \leq w \\ f(w, i+1) & \text{else} \end{cases}$$

Base Case :-

$$\forall i, f(0, i) = 0$$

for i in N :

for w in $[1, \dots, K] \rightarrow$ This is wrong, use $[x, \dots, 1]$ instead

$$dp(w) = dp(w - a_i)$$

Shortest Reliable Paths

Input :- Edge wtd graph s, t & $k \in \mathbb{N}$

Output :- The shortest path from s to t in G using $\leq k$ hops/edges

Subproblems :-

$$\forall x \in V, \forall i \leq k$$

let $\text{dist}(u, i)$ be the length of the shortest path from s to u , using i hops.

Main Problem :- $\min_{i \leq k} \text{dist}(t, i)$

Recurrence Relation :-

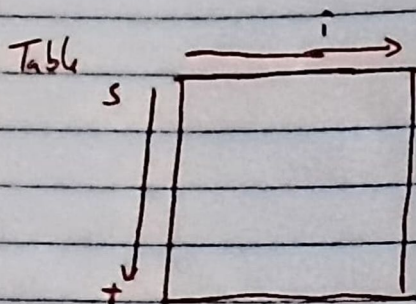
$$\text{dist}(u, i) = \min_{(v, u) \in E} (\text{dist}(v, i-1) + w(v, u))$$

Exist Subproblems:-

$$\text{dist}(s, s) = 0$$

$$\text{dist}(s, \infty) = \infty$$

$$\text{dist}(u, s, s) = \infty$$



All pair shortest paths

Input:- G

Output:- $\forall u, v$, length of shortest path from u to v

Subproblems:- let $\text{dist}(u, v, k)$ be the length of the shortest path from u to v using intermediate nodes amongst $[1, k]$

Main Problem :-

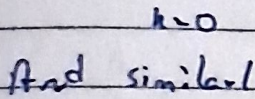
$$k = |V|$$

Recurrence Relation :-

$$\text{dist}(u, v, k) = \min(\text{dist}(u, v, k-1), \text{dist}(u, k, k-1) + \text{dist}(k, v, k-1))$$

Easiest Subproblem

$$\text{dist}(u, v, 0) = \begin{cases} w(u, v) & \text{if } (u, v) \in E \\ \infty & \text{else} \end{cases}$$



↓

Same as n dijkstras with fib heaps

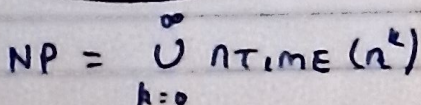
$$P_{\text{class } P} = \bigcup_{k=0}^{\infty} \text{TIME}(n^k)$$

↳ All problems solved by a DTM in polynomial time

CLIQUE = $\{ \langle G, k \rangle \mid G \text{ has a clique of size } k \}$

NP $\rightarrow$ Problems whose solutions can be verified in polynomial time

IP $\rightarrow$ Interactive Proofs $=$ PSPACE $=$ NPSPACE



$$L(v) = \{x \mid \exists n, V(z, n) \text{ accepts } x\}$$

V verifies L if $L(V) = L$

$$\text{NTIME}(n) = \{ L \mid L \text{ is decided by some NDTM } M \text{ in time } O(n) \}$$

NP = $\{L \mid L \text{ has an efficient verifier}\}$

NP Completeness

Theorem:- Boolean satisfiability is NP-complete
 $\hookrightarrow \text{SAT}$

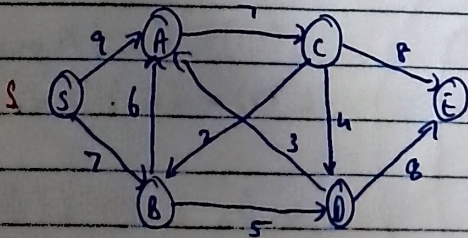
Polynomial Time Redn $L_1 \leq_p L_2$ if $\exists f$ computable in polynomial time and
 $x \in L_1 \iff f(x) \in L_2$

All problems in NP are polynomial time reducible to the class NP Complete

SAT = $\{\phi \mid \phi \text{ is a CNF Boolean formula that is not a tautology}\}$

CLIQUE $\in$ NPC

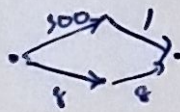
Linear Programming
Network Flows



What is the max flow from S to E?

Kirchoff's laws hold here.

$\sum \text{in} = \sum \text{out}$



flow $f: E \rightarrow \mathbb{R}$

$$\forall e \in E \quad f(e) \leq c(e)$$

$\hookrightarrow$ capacity (edge wts)

Objective:-

Maximize flow

$$\text{Max} \sum_{e=(u,v) \in E} f(e)$$

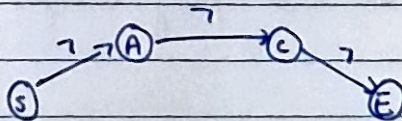
Thm:- Max flow = Minimum cut

Max flow $\leq$ Minimum cut trivially

Ford Fulkerson Algorithm:-

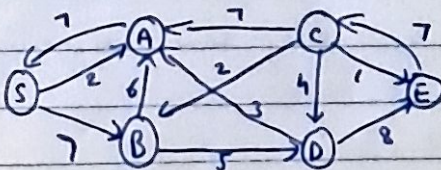
$\rightarrow$ Current flow

$\rightarrow$ Augment the flow $\rightarrow$ find an augmenting path

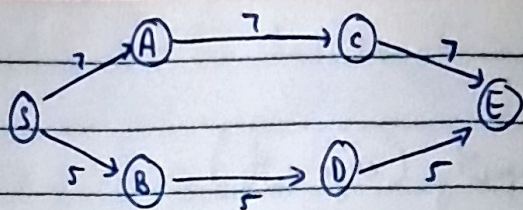


Residual Graph Capacities given current flow f

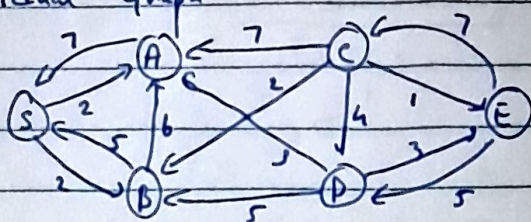
$c(e) - f(e)$ in the original direction
 $f(e)$ in the reverse direction



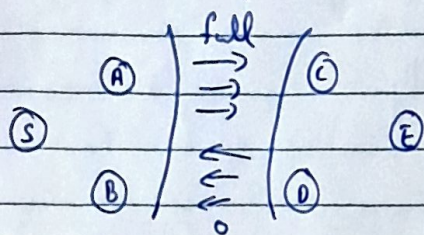
Next augmenting path $\rightarrow$ SBDE. Add it to the original augmented path and repeat.



Residual Graph



Here, $\exists$ no path from $S \rightarrow E$



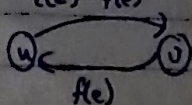
in the original graph

This implies that max flow = $\sum$ capacity of edges b/w their cuts.

Input:- $G = (V, E)$, edge weighted graph

Output:- $f: E \rightarrow \mathbb{R}$ max flow
conserved & valid / feasible

Algo:- Augment current flow by adding some augmenting path in/from the residual graph



Approximation Algorithm :-

Example :- Minimum Set Cover

Input: A set S and a family of subsets of S , $\mathcal{F} \subseteq 2^S \equiv \mathcal{P}(S)$ → Must be finite

Output: The least number of elements of $\mathcal{F}$ that cover S [i.e. the union S]

Facts:-

- (i) Minimum Set Cover is NP-Hard
- (ii) There is a greedy strategy that approximates the answer within a factor of $\log |S|$. i.e. computed answer $\leq \log |S|$
↳ Approximation Ratio.

Example:-

$$S = \{1, 2, 3, 4\}$$

$$\mathcal{F} = \{\{1, 2\}, \{1, 2, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}\}$$

$$MSC = 2$$

Greedy Algorithm:-

Always choose the next element of $\mathcal{F}$ that covers the maximum number of not yet covered elements of S .

In the above example, it would return $\{1, 2, 3\}, \{1, 4\}$
 → $MSC = 2$

Theorem:- Approximation Ratio is $\log |S|$ (of Greedy Set Cover)
 (or) The obtained answer $\leq \log |S|$

To show:- $\tau \leq \log |S| \cdot k$

Proof: Let n_i be the no. of uncovered elements in S after i iterations of the greedy algo.
 $\{n_0\} = |S|$; $n_i < 1$

After i iterations,

n_i : remains uncovered
 : $\exists$ k sets that cover it.

Using pigeonhole principle

$\exists$ a set that covers $\geq \frac{n_i}{k}$

$$\Rightarrow n_{i+1} \leq n_i - \frac{n_i}{k}$$

$$\Rightarrow n_{i+1} < n_i \left(1 - \frac{1}{k}\right)$$

$$n_i \leq n_0 \left(1 - \frac{1}{k}\right)^i$$

$$n_i \leq |S| \left(1 - \frac{1}{k}\right)^i$$

$$1 - x \leq e^{-x} \quad \forall x \in [0, 1]$$

$$1 - \frac{1}{k} \leq e^{-1/k}$$

$$n_i \leq |S| e^{-i/k}$$

$$i = k \ln |S|$$

$$n_{k \ln |S|} \leq |S| e^{-\frac{k \ln |S|}{k}} = 1$$

After $t = (\ln |S|) k$ iterations, q

$SAT = \{\phi \mid \phi \text{ is a satisfiable CNF Boolean formula}\}$

$\hookrightarrow$ Conjunctive Normal Formula

$$\phi = \bigwedge (\bigvee x_i)$$

DNF $\rightarrow$ Disjunctive Normal Formula.

The Cook-Levin Theorem states that SAT is NP-complete

$\hookrightarrow$

$$SAT \in NP$$

$$\forall L \in NP ; L \leq_p SAT$$

$L \in NP \rightarrow \exists$ NDTM N that decides L in polynomial time.

Let ϕ_N be CNF

ϕ_N is satisfiable iff N has an accepting path.

Karp Redⁿ $\rightarrow$ Poly time mapping redn

$A \leq_p B$ if $\exists$ polynomial time computable f s.t.
 $x(A) = f(x(B))$.

Theorem:- CLIQUE is NP-Complete

Proof:- Recall $CLIQUE \in NP$

$CLIQUE = \{ \langle G, k \rangle \mid G \text{ has a CLIQUE of size } k \}$

Trivially $CLIQUE \leq_p SAT$

We must show $SAT \leq_p CLIQUE$

$SAT \leq_p 3-SAT \rightarrow \{\phi \mid \phi \text{ is a satisfiable 3-CNF}\}$

$$(x_1 \vee x_2 \vee x_3 \vee x_4) = (x_1 \vee x_2 \vee x_3) \wedge (x_3 \vee x_4 \vee \bar{x}_3)$$

This can be applied recursively for larger ~~n~~ n-SATs.

Given a 3-CNF ϕ
 n variables & l clauses

Construct a graph G_ϕ

Create 3 nodes per clause $\rightarrow 3l$ vertices.

Do not add edges within clauses. Instead add
 non contradictory edges across clauses.
 $\hookrightarrow$ There shouldn't be a label mismatch along the
 edge [no edge b/w x_i & $\bar{x}_i$]

Does G_ϕ have a clique of size l .

Assume the 3-SAT has a satisfying assignment $\rightarrow$
 at least 1 T in each clause.

Pick the first T from each clause $\rightarrow l$ literals
 - l vertices that are not contradictorily labelled $\rightarrow$
 $\exists$ a clique of size l .

Suppose G_ϕ has a clique of size $l \rightarrow$ result of picking
 1 node from each clause [within a clause $\rightarrow$ no edge]
 We can then say that these were all T since they
 are non contradictorily labelled [works for F, think].

Hence since SAT $\leq_P$ CLIQUE & vice versa,
 it is NP-complete.